
AcceRL: A Distributed Asynchronous Reinforcement Learning and World Model Framework for Vision-Language-Action Models

Chengxuan Lu*, Shukuan Wang*, Yanjie Li, Yingying Fang, Huoyan Wang, Tian Zhang, Wei Liu, Shiji Jin, Fuyuan Qian, Peiming Li, Chao Xu, Baigui Sun,† Yang Liu†

IROOTECH TECHNOLOGY

Wolf 1069 b Lab, Sany Group

{chengxuan.lu, shukuan.wang, yanjie.li, yingying.fang,
huoyan.wang, tian.zhang, wei.liu, shiji.jin, fuyuan.qian,
peiming.li, chao.xu, baigui.sun, yang.liu1}@irootech.com

Abstract

Reinforcement learning (RL) for large-scale Vision-Language-Action (VLA) models is severely bottlenecked by synchronization barriers and the high cost of environment data acquisition. To overcome these challenges, we propose AcceRL, a distributed asynchronous RL framework that physically isolates environment roll-outs, model inference, and gradient updates. By eliminating the cascading long-tail idle bubbles inherent in synchronous systems, AcceRL maximizes hardware utilization and ensures scalable throughput. Furthermore, AcceRL features a modular design that supports the integration of diverse, plug-and-play world models into its distributed pipeline. Extensive experiments demonstrate that the base framework achieves highly competitive performance across all four LIBERO [1] task suites. Systematically, the asynchronous architecture delivers a $2.4\times$ throughput speedup over leading synchronous baselines. Algorithmically, by leveraging a world model pre-trained on 1,000 offline trajectories, AcceRL achieves up to a $200\times$ improvement in online sample efficiency on LIBERO-Spatial, establishing a robust framework that is both sample-efficient and time-efficient for embodied AI. Code is included in the supplementary material. Code is available at <https://github.com/distanceLu/AcceRL>.

1 Introduction

Vision-Language-Action (VLA) models [2–4] have rapidly become the default substrate for general-purpose embodied agents, executing complex tasks directly from natural-language instructions. While imitation learning [5, 6] has driven much of the recent progress, it remains limited by poor generalization and error accumulation under covariate shift. The community has therefore turned to reinforcement learning (RL) to optimize long-horizon policies and perform online fine-tuning of pre-trained VLA backbones. Effective RL on VLA models, however, requires massive parallelization to generate the interaction trajectories needed for stable learning, posing an urgent need for an efficient, large-scale RL framework for VLA.

Developing such a framework faces severe challenges in two fundamentally different ways. First, on the efficiency axis, large-scale distributed RL training for VLA models is highly system-intensive. Simulator iteration, multimodal model inference, and parameter updates compete for the same GPU

*Equal contribution.

†Corresponding author.

resources, and any pacing mismatch among them causes pipeline stalls. Most existing synchronous frameworks [7, 8] suffer from severe long-tail phenomena that compound this problem. Recent asynchronous frameworks for VLA-RL (e.g., RL-VLA³ [9]) make substantial progress, but their efficiency gains heavily rely on vectorized simulators. While effective on highly parallel benchmarks, this premise breaks down in real-world physical robot deployments, simulators resisting parallelization, or mixed training across heterogeneous tasks. In such regimes, where natural batchability is absent, they struggle to hide environment latencies, leading to severe idle bubbles and a significant drop in throughput. Second, on the data axis, large VLA models are extremely data-hungry. Scaling RL to real-world physical robots faces severe data-collection bottlenecks. Introducing a learned World Model (WM) to synthesize experience naturally resolves this physical sampling limit, paving the way for real-world adaptability. Yet integrating it into a high-performance distributed RL pipeline remains unsolved. Existing WMs (like the Dreamer family [10]) tightly couple policy and dynamics within shared latent spaces, severely hindering the plug-and-play integration of independent pre-trained models. Conversely, pixel-level video WMs [11, 12] provide a naturally decoupled visual interface but have only served as standalone simulators, never fully integrated into an asynchronous RL loop.

To bridge this gap, we introduce AcceRL, a modular and fully asynchronous framework tailored for large-scale VLA models. AcceRL achieves complete system decoupling by physically isolating training, inference, and sampling into independent asynchronous streams that communicate through shared buffers, without any batchability or vectorization assumptions on the producer side. Furthermore, it pioneers the integration of a pixel-level trainable world model as a co-equal experience source alongside real-environment workers, bridged by a value recomputation mechanism. Our key contributions are as follows:

- **High-throughput asynchronous RL architecture:** AcceRL realizes full asynchrony among rollout, inference, and training. By removing synchronization barriers, eliminating frequent mode-switching, and mitigating long-tail latencies, it maximizes hardware utilization. Crucially, unlike prior asynchronous frameworks dependent on vectorized simulators, AcceRL sustains consistently high throughput and resource efficiency even in complex environments where natural batchability is absent, achieving a $2.4\times$ throughput speedup over leading synchronous baselines.
- **Pioneering integration of a trainable world model:** AcceRL pioneers the integration of a pluggable, pixel-level trainable world model into a distributed asynchronous RL pipeline. The world model operates through a pixel-space interface, which helps preserve the simulator’s observation-action structure and facilitates compatibility with different VLA backbones. To absorb the compound staleness arising from full asynchrony, we combine an optimized value recomputation mechanism with the adopted GIPO [13] algorithm. By leveraging a world model pre-trained on 1,000 offline trajectories (which are out-of-distribution and more cost-effective to collect than online samples) and breaking through physical sampling limitations via imagination learning, AcceRL improves online sample efficiency by up to $200\times$ over a strong asynchronous model-free baseline on LIBERO-Spatial [1] under matched compute.
- **Empirical effectiveness and competitive performance:** AcceRL achieves highly competitive performance on the LIBERO benchmark, attaining the best results among the compared methods on most task suites while demonstrating outstanding system scalability, high computational efficiency, and rapid convergence with high data efficiency.

2 Related work

2.1 Evolution of Distributed Reinforcement Learning Frameworks

Historically, scaling RL relies on synchronous distributed frameworks. For Large Language Models (LLMs), frameworks like OpenRLHF [14] and DeepSpeed Chat [15] combine high-performance inference engines with distributed training backends to optimize memory during synchronous parameter updates. Similarly, the veRL framework [16] adopts a hybrid flow architecture that separates control logic from tensor operations while maintaining lockstep synchronization between generation and training. This paradigm naturally extends to the VLA domain. Systems such as siRL [17] and RLLAVA [18] use colocated execution with synchronized actor-learner resource sharing. RLinf-VLA [19] further optimizes this with a hybrid fine-grained pipeline to overlap rendering and training,

but retains the core synchronous stepping mechanism. Despite mathematical stability, synchronous architectures suffer from long-tail effects: the central learner waits for all distributed Rollout Workers to finish trajectory collection before gradient updates [20], which maintains the bounded policy lag required by PPO [21]. LLM post-training has two-level long-tail latency from episode-level token generation and cluster-level from network communication. Moreover, VLA frameworks have a third more complex long tail. Because physical simulators require dynamic computational overhead, the entire distributed system is forced to idle while waiting for the slowest physics step.

To mitigate synchronization overhead and pipeline bubbles, the field has shifted to asynchronous/decoupled architectures. For LLMs, frameworks decouple generation and training: Laminar [22] uses trajectory-level asynchrony for long-tail text generation. AReaL [23] fully decouples them to reduce GPU underutilization from reasoning tokens. DistFlow [17], LlamaRL [24] and ROLL [25] also adopt decoupled/asynchronous strategies, but these are tailored for in-memory closed-loop autoregressive generation, misaligned with embodied agents’ external dependencies. RL-VLA³ [9] proposes triple-level decoupling for embodied AI but is not fully asynchronous, failing in complex physical scenarios. To overcome this physical wall-clock bottleneck, the system needs to achieve complete physical-logical decoupling. AcceRL accomplishes this fully asynchronous model by entirely separating the training, inference, and rollout.

2.2 World Models and Model-Based RL

Model Based Reinforcement Learning fundamentally accelerates convergence by learning a predictive model of environment dynamics to substitute costly physical interactions with computational predictions. A landmark in this field is the Dreamer series [10] which projects high dimensional observations into a compact discrete latent space. By optimizing policies within this differentiable dream using Recurrent State Space Models can achieve high sample efficiency in environments with dense rewards such as Atari games.

While latent space models are efficient for coarse grained tasks transitioning to pixel level world models based on Diffusion Models [26] becomes a structural necessity for embodied VLA models. Pixel-level prediction retains fine-grained visual context critical for learning complex contact dynamics. Using raw pixels as a universal interface enables policy-world model decoupling for independent scaling and integration, free of shared latent encoder constraints. Additionally, diffusion models avoid complex joint loss optimization in heterogeneous model grafting, offering a stable, high-fidelity virtual sandbox for fine-tuning pre-trained models. Building on these pixel level predictions frameworks such as Genie [11] and DIAMOND [12] have demonstrated the potential of world models as infinite risk free simulators. However integrating these models into VLA training loops introduces severe VRAM bottlenecks. Recent works like WoVR [27] attempt to bridge this gap but rely on a collocated GPU allocation strategy. This necessitates frequent offloading and onloading of massive parameters between CPU and GPU leading to significant compute idleness and low throughput. By physically isolating policy and world models on dedicated hardware AcceRL eliminates memory swapping and reduces communication to trajectory passing, which supports independent dynamics control and resource reuse.

3 AcceRL: A Fully Asynchronous Training Framework

As shown in Figure 1, traditional synchronous RL frameworks suffer from heavy synchronization overhead caused by a cascading long-tail effect across three distinct levels:

- **Step-Level Long-Tail:** Physical simulation times vary across environment instances. Synchronous setups force the GPU to wait for the slowest worker to finish a step before executing batched inference, causing step-level idle bubbles.
- **Episode-Level Long-Tail:** Episodes end at different times due to varying termination conditions. Synchronous pipelines prevent the GPU from starting new episodes until all parallel workers finish, forcing early-finishing environments into idleness.
- **Cluster-Level Long-Tail:** Global synchronization barriers require all GPUs to complete their rollouts before executing a gradient update. This caps the entire cluster’s throughput at the slowest GPU, creating massive system-wide bubbles.

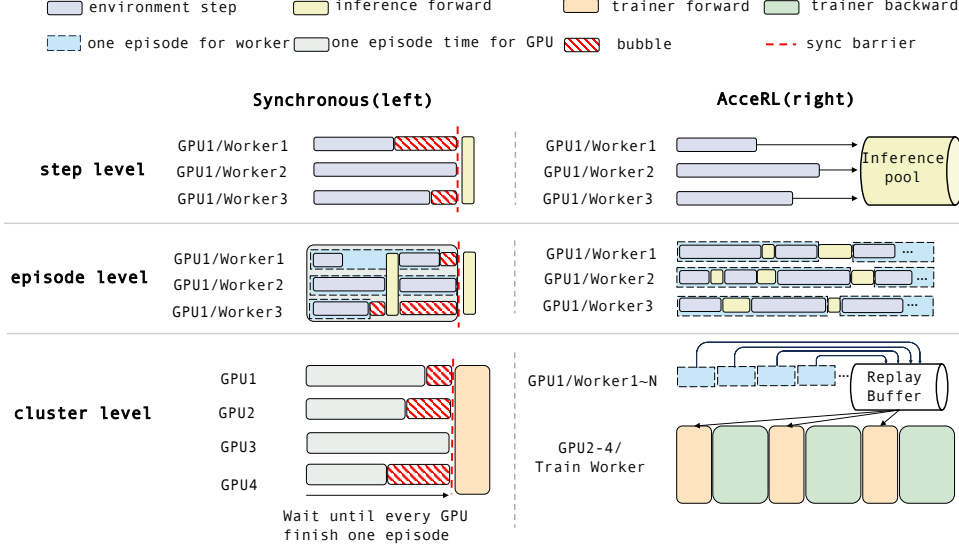


Figure 1: **Timelines and GPU bubbles across three levels of synchronous RL framework (left) and the asynchronous framework AcceRL (right).** The synchronous framework suffers from long-tail effects at the step, episode, and cluster levels, resulting in severe forced waiting bubbles (red hatched areas). By introducing a dual asynchronous mechanism, AcceRL substantially reduces synchronization barriers and long-tail idle periods, enabling more efficient computation.

As training scales to multi-billion parameter VLA models across large clusters, these three layers of latency compound exponentially, leading to prohibitive hardware underutilization.

To overcome these bottlenecks, we propose AcceRL (Figure 2a), a high-throughput asynchronous framework designed for large-scale Embodied AI. The core advantage of AcceRL is the complete decoupling of training, rollout, and inference for asynchronous parallel execution. It utilizes the NCCL weight synchronization mechanism, a non-blocking FIFO replay buffer mechanism, and a data prefetching mechanism to improve training efficiency (see the appendix D for details).

3.1 Macro-Asynchrony: Decoupling Training and Rollout

AcceRL eliminates cluster-level synchronization overhead by fully decoupling Rollout and Trainer Workers into a non-blocking circular pipeline, bypassing the strict lockstep or bounded staleness of prior frameworks [23]. As illustrated in Figure 2a, Rollout Workers execute interruptible rollouts, streaming trajectory segments immediately to a FIFO distributed replay buffer $\mathcal{B}$. Concurrently, Trainer Workers sample batches from $\mathcal{B}$ for continuous parameter updates. To address the resulting policy lag from stale data without sacrificing throughput, we utilize value recomputation and GIPO (Section 5).

For large-scale efficiency, Trainer Workers employ ZeRO-2 [28] to partition optimizer states and gradients, supporting larger micro-batch sizes. Updated parameters are subsequently synchronized to Inference Workers via NCCL broadcast. This architecture ensures maximum-speed training and significantly reduces end-to-end wall-clock time.

3.2 Micro-Asynchrony: Decoupling Interaction and Inference

While macro-asynchrony removes global barriers, the second level of asynchrony eliminates micro-level idle bubbles caused by step- and episode-level long-tails. Traditional tightly coupled simulation and inference suffer from severe straggler effects and poor GPU batching due to highly variable environment step times and episode horizons. To overcome this problem, AcceRL decouples physical interaction from policy inference via an Inference-as-a-Service paradigm.

Rollout Workers manage environment instances. After generating an observation o_t , they send asynchronous requests to a centralized inference pool and then suspend immediately to avoid local

computation delays. Every Inference Worker maintains a request queue Q and trigger batched forward passes using a dynamic window mechanism to balance GPU utilization and latency:

$$\text{Trigger} = (|Q| \geq B) \vee (t_{\text{now}} - t_{\text{first}} \geq T_{\text{max}}) \quad (1)$$

where B is the target batch size, t_{now} is the current timestamp, and T_{max} is the maximum allowable wait time since the oldest request's arrival t_{first} . Computed actions are then returned to the respective workers.

After each episode terminates, completed sequences are packaged as trajectories τ and stored in the replay buffer $\mathcal{B}$:

$$\tau = (o_{1:T+1}, a_{1:T}, r_{1:T}, \mu_{1:T}, v_{1:T}, \tilde{v}_{T+1}, \text{done}), \quad (2)$$

where o_t , a_t , and r_t denote standard observations, actions, and rewards; μ_t represents the behavior policy probabilities for off-policy correction; v_t and $\tilde{v}_{T+1}$ are state and bootstrapped value estimates; and done indicates natural episode termination.

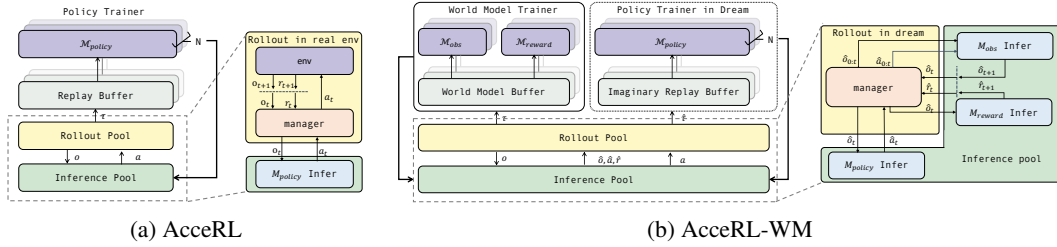


Figure 2: **The pipeline of AccerL base backbone (left) and world-model-augmented mode (right).** The framework isolates training, inference, and rollout into independent asynchronous streams. By integrating a plug-and-play world model, AccerL bypasses physical simulation constraints through imagination rollouts to significantly enhance sample efficiency.

4 AccerL with World Model

While AccerL maximizes hardware utilization, direct physical environment interaction inherently bottlenecks sample efficiency. To overcome this, the framework introduces an augmented mode, AccerL-WM, which integrates a plug-and-play world model to enable rapid, sample-efficient imagination rollouts. This world model comprises an observation model M_{obs} to simulate dynamics and a reward model M_{reward} to evaluate success. As illustrated in Figure 2b, AccerL-WM deploys independent Trainer and Inference Workers for these models and partitions the distributed replay buffer into $\mathcal{B}_{\text{wm}}$ for real-world state transitions and $\mathcal{B}_{\text{img}}$ for synthetic trajectories.

4.1 Alternating Rollout and Learning in Imagination

As Figure 2b shown, to generate data for the world model and policy, Rollout Workers use an alternating sampling strategy. A real trajectory τ is first collected from the real-world environment and serves two purposes. It is added to $\mathcal{B}_{\text{wm}}$ as training data for the world model, and cached locally to supply initial grounding states for imagination rollouts.

Following the real rollout, the worker generates numerous imagined trajectory episodes, denoted with a hat $\hat{\cdot}$. The imagination pipeline operates as follows. First, a real frame o_t is sampled from the local episode buffer as the initialization and starting state, where $\hat{o}_t = o_t$. Next, the worker invokes M_{policy} to produce the imagined action $\hat{a}_t$. Then, it queries M_{obs} to generate a predicted next frame $\hat{o}_{t+1}$. Meanwhile, the predicted frames are fed into M_{reward} to estimate the success probability of the current state and output the corresponding termination signal $\hat{\text{done}}$.

Crucially, to mitigate the compounding errors inherently caused by long-term autoregressive generation, these imagined trajectories are strictly restricted to a fixed horizon H rather than building full executions. The resulting short-term trajectory episode $\hat{\tau}$ is added to $\mathcal{B}_{\text{img}}$:

$$\hat{\tau} = (\hat{o}_{t:t+H+1}, \hat{a}_{t:t+H}, \hat{r}_{t:t+H}, \hat{\mu}_{t:t+H}, \hat{v}_{t:t+H}, \tilde{v}_{t+H+1}, \hat{\text{done}}) \quad (3)$$

For these imagined transitions, AcceRL utilizes a potential-based reward structure formulated as:

$$\hat{r}_\tau = M_{reward}(\hat{o}_{t+1}) - M_{reward}(\hat{o}_t) \quad (4)$$

where $\hat{o}_t$ denotes the synthesized observation at step t , which can be a multi-frame stack to capture temporal dynamics. By defining the reward as the marginal difference in success probability between successive synthetic states, this formulation helps preserve policy invariance. It accelerates training by providing dense, informative guidance without altering the fundamental objective of the original manipulation task.

4.2 Training pipeline of AcceRL-WM

Rather than relying on a static pre-trained world model, AcceRL-WM continuously fine-tune both M_{obs} every T_{obs} cycles and M_{reward} every T_{reward} steps. This constant grounding ensures the dream space adapts to novel states explored by the evolving policy. During the training phase, the Trainer Worker manages three fully independent concurrent optimization loops with different update frequencies to stabilize M_{policy} , M_{obs} , and M_{reward} .

- M_{policy} Optimization: The policy trainer continuously samples imagined data batches $\hat{\tau}$ from the imaginary replay buffer $\mathcal{B}_{img}$ to update M_{policy} , completely bypassing the physical simulator’s latency.
- M_{obs} Optimization: The observation model is fine-tuned every T_{obs} cycles using the real-world trajectory data sampled from $\mathcal{B}_{wm}$, ensuring the dream space remains visually and physically accurate.
- M_{reward} Optimization: The reward model performs regression on real (o_t, r_t) pairs sampled from $\mathcal{B}_{wm}$ every T_{reward} steps to refine its success probability estimations.

To minimize communication overhead, updated weights are broadcasted to their respective nodes in the Inference Pool only when an actual update occurs.

5 Mitigating Policy Lag in Asynchronous Training

While AcceRL’s fully asynchronous and decoupled architecture maximizes hardware utilization, it introduces policy lag. The macro-asynchrony between the Rollout Workers and Trainer Workers causes the behavior policy μ used during data collection to become stale compared to the continuously updating learner policy π . This staleness introduces off-policy bias. To mitigate the off-policy bias induced by policy lag, the Trainer Worker employs a dual mechanism: it utilizes value recomputation for real-time advantage correction to eliminate staleness in value estimation, and applies GIPO [13] for robust gradient calibration to stabilize policy updates against delayed trajectories.

Value Recomputation: In fully asynchronous RL frameworks, policy lag introduces a certain degree of off-policy bias, necessitating state value recomputation to realign advantage estimation with the current policy. Traditionally, this requires an additional full forward pass over the dataset, severely bottlenecking throughput. Furthermore, global random data shuffling and synchronous aggregation for advantage statistics introduce memory fragmentation and communication latency.

To resolve these bottlenecks, we propose a low-overhead pipeline. Instead of a separate preprocessing phase, we push Generalized Advantage Estimation (GAE) [29] directly to the post-forward-propagation stage of micro-batch training. Given our single-epoch update design, these on-the-fly advantage estimates closely approximate those obtained by forced re-inference, enabling timely value correction while avoiding the substantial cost of additional policy forward passes. At the data level, we replace global shuffling with sequential micro-batch slicing, ensuring contiguous memory access. Finally, we mask communication latency via asynchronous lag normalization: the system uses the previous step’s global moving statistics for current normalization, deferring synchronous aggregation to the end of backpropagation.

By eliminating redundant inference, streamlining memory access, and deeply overlapping communication with computation, these optimizations achieve a 30% increase in end-to-end training speed. They effectively mitigate policy lag without compromising convergence. Mathematical equivalence proofs and implementation details is provided in Appendix C.1.

Gradient Calibration (GIPO): Even with updated advantages, extreme policy lag can cause massive divergence if standard PPO is used, as its hard clipping mechanism zeroes out gradients for highly stale data, stalling the learning process. To protect the policy, AcceRL employs Gaussian Importance sampling Policy Optimization (GIPO) [13]. Let $\rho_t(\theta) = \pi_\theta(a_t | o_t) / \mu(a_t | o_t)$ denote the importance ratio between the learner and behavior policies, and $\bar{\rho}_t$ be its stop-gradient version. GIPO introduces a Gaussian trust weight, governed by hyperparameter σ , to softly damp extreme ratios:

$$\omega(\bar{\rho}_t; \sigma) = \exp\left(-\frac{1}{2} \left(\frac{\log(\bar{\rho}_t)}{\sigma}\right)^2\right) \quad (5)$$

$$\mathcal{L}_\pi^{\text{GIPO}}(\theta) = -\mathbb{E}_{(o_t, a_t) \sim \mathcal{B}} \left[\omega(\bar{\rho}_t; \sigma) \cdot \rho_t(\theta) \cdot A_t \right] \quad (6)$$

where A_t represents the advantage estimate. This smooth, continuous penalty inherently bounds the policy update magnitude without abruptly discarding useful gradients, allowing the framework to safely extract learning signals from highly delayed trajectories.

6 Experiments

We evaluate our framework across two diverse simulated domains: the comprehensive LIBERO benchmark [1] (encompassing the Spatial, Object, Long, and Goal suites) and contact-rich continuous control tasks in ManiSkill [30].

6.1 Throughput Comparison

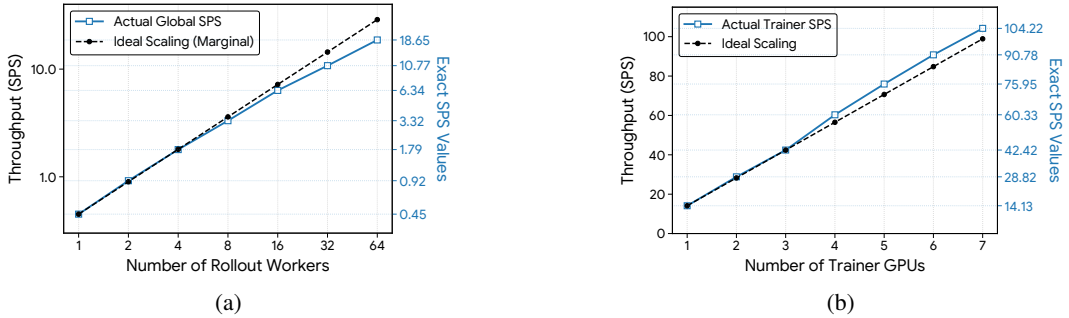


Figure 3: **Scalability performance of the AcceRL framework with decoupled data labels.** (a) Throughput scaling demonstrates near-linear performance up to 64 Rollout Workers. (b) Trainer scalability shows that the actual trainer SPS closely tracks the ideal marginal scaling curve when scaling up to 7 GPUs. Both configurations effectively maintain high hardware utilization without suffering from severe communication overhead.

We first evaluate the training throughput and scalability of AcceRL. By decoupling environment interaction from action generation and using centralized dynamic batching, AcceRL effectively reduces GPU bubbles caused by long-tail inference latency. To demonstrate the efficiency of this design, Figure 3 shows the scaling performance of the system. Notably, when scaling the trainer to 7 GPUs(H200), AcceRL exhibits a super-linear scaling effect, reaching 104.22 SPS (samples per second). This super-linear scaling is primarily driven by DeepSpeed ZeRO-2 [31]. As the cluster expands, it enables the system to accommodate a larger micro-batch size per GPU without out-of-memory errors. The increased batch size heavily amortizes kernel launch overheads and maximizes Tensor Core utilization, allowing the actual throughput to consistently surpass the ideal marginal scaling curve while maintaining a high GPU utilization of over 94%. Moreover, Table 1 highlights how this near-saturated utilization ($\sim 94\text{--}95\%$) creates a stark contrast with existing baselines. By circumventing the severe offloading overheads and physical simulator bottlenecks that limit SimpleVLA ($\sim 30\text{--}40\%$ utilization) and RLinf ($\sim 50\text{--}60\%$ utilization), AcceRL delivers a superior and stable sampling throughput of 42.4 SPS. Ultimately, this allows AcceRL to reach 2,000 training steps in just 6.5 hours delivering a 2.4 times speedup over RLinf and nearly a 2.6 times speedup over the synchronous SimpleVLA baseline.

Table 1: **Comparison of system efficiency and training throughput on $4\times$ H200 GPUs.** Under the same $4\times$ H200 hardware setting, AcceRL achieves higher trainer GPU utilization and sampling throughput than the compared baselines, substantially reducing the estimated wall-clock time required to reach 2K training steps.

Framework	Peak Trainer GPU Util.	Throughput (SPS) @ 90% Succ.	Throughput (SPS) @ 99% Succ.	Est. Time to 2K Steps
SimpleVLA (VeRL)	$\sim 30\text{--}40\%$	~ 21.4	~ 28.5	~ 16.7 hrs
RLinf	$\sim 50\text{--}60\%$	~ 25.2	~ 35.6	~ 15.8 hrs
AcceRL (Ours)	$\sim \mathbf{94\text{--}95\%}$	$\sim \mathbf{42.4}$	$\sim \mathbf{42.4}$	$\sim \mathbf{6.5}$ hrs

6.2 Task Performance on LIBERO

We evaluate AcceRL against the OpenVLA-OFT supervised baseline [32] and recent state-of-the-art RL frameworks (RLinf-VLA [19], SimpleVLA-RL [33]) across four diverse LIBERO task suites.

Table 2: **Task performance comparison on the LIBERO benchmark.** AcceRL achieves superior success rates across most categories. Notably, it attains a 99.1% success rate on Long tasks, demonstrating stability in long-horizon scenarios where supervised fine-tuning typically falters.

Method	Spatial (%)	Object (%)	Goal (%)	Long (%)
Ours (AcceRL)	99.6	100.0	98.8	99.1
SimpleVLA-RL [33]	99.4	99.8	99.2	98.5
RLinf-VLA [19]	99.4	99.8	98.8	94.0
OpenVLA-OFT	96.2	98.3	96.2	90.7

As summarized in Table 2, AcceRL achieves the best or tied-best performance on most task suites and remains competitive on Goal. OpenVLA-OFT struggles on LIBERO-Long tasks (90.7%) due to compounding errors inherent in supervised imitation learning. While recent RL frameworks improve upon this supervised baseline, they still exhibit performance degradation in demanding long-horizon scenarios. In contrast, by formulating the manipulation problem through a distributed RL paradigm, AcceRL explicitly maximizes long-term rewards rather than mimicking single-step actions. This mechanism allows the policy to recover from minor deviations, ensuring stable execution throughout complex multi-stage tasks and achieving a striking 99.1% success rate in the Long category.

6.3 Task Performance on ManiSkill

To evaluate AcceRL in complex, contact-rich continuous control tasks, we conducted experiments on the ManiSkill PickCube environment (detailed configurations and hyperparameters are provided in Appendix E, Table 3), which demands precise physical interaction and dynamic feedback.

As shown in Figure 4a, the supervised fine-tuning baseline, OpenVLA-OFT, tends to plateau at a success rate of 66%–80% under our ManiSkill PickCube setting. In contrast, AcceRL reaches a peak success rate of approximately 90% within 60k training steps, showing clear gains over supervised fine-tuning. This improvement highlights the benefit of online RL for contact-rich manipulation, where policies must recover from compounding errors and adapt to physical interaction feedback. Rather than merely imitating isolated single-step actions, AcceRL directly optimizes long-horizon task rewards through distributed environment interaction.

6.4 Performance Evaluation of AcceRL-WM on the LIBERO

To evaluate the learning efficiency of our model-based architecture, we initialize the policy with a suboptimal checkpoint (pre-trained on limited demonstrations). As illustrated in Figure 4b, integrating the DIAMOND [12] world model, which is pre-trained on 2,000 out-of-distribution offline trajectories, drastically accelerates VLA fine-tuning. The learning curves demonstrate that AcceRL with the world model requires significantly fewer real environment interactions to achieve a high average return compared to the model-free AcceRL baseline. Specifically, the horizontal gap on the logarithmic

x-axis reveals an approximately $200\times$ (20,000%) improvement in online sample efficiency. This massive reduction in physical sampling cost is fundamentally driven by the world model’s ability to substitute costly real-world interactions with extensive imagination rollouts. Furthermore, leveraging dense potential-based reward signals computed for every imagined transition effectively alleviates the sparse feedback typical of traditional RL and provides an informative optimization landscape.

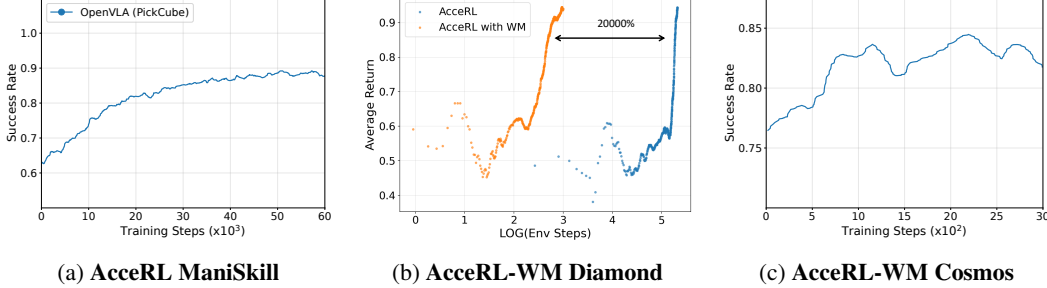


Figure 4: (a) On ManiSkill PickCube, AcceRL reaches a $\sim 90\%$ peak success rate, overcoming the early plateaus of supervised baselines. (b) On LIBERO-Spatial, AcceRL-WM demonstrates substantially higher sample efficiency than the model-free AcceRL baseline by reaching a comparable average return with far fewer real environment interactions, corresponding to an approximately $200\times$ reduction in required environment steps. (c) Replacing the DIAMOND world model with Cosmos, AcceRL-Cosmos successfully completes the closed-loop imagined rollout and policy-update pipeline, suggesting that AcceRL can accommodate alternative world-model backends with limited engineering changes.

6.5 Generalizing AcceRL to the Cosmos World Model

To further validate the pluggability of AcceRL, we replace the original DIAMOND world model with the Cosmos diffusion world model [34] while keeping the OpenVLA-OFT policy and the main RL training pipeline unchanged. Given historical multi-view observations and future action sequences, Cosmos predicts subsequent visual states, enabling pixel-space imagined rollouts for policy fine-tuning.

Since Cosmos introduces higher computational cost and different dependency requirements, we deploy its components in isolated Ray [8] actors and schedule them within the same distributed AcceRL framework. As shown in Figure 4c, AcceRL-Cosmos successfully completes imagined rollouts and subsequent policy updates on LIBERO-Spatial, demonstrating that AcceRL can flexibly support heterogeneous world model backends with minor pipeline modification.

7 Conclusion

This paper presents AcceRL, an asynchronous reinforcement learning framework optimized for large scale VLA. The core strength of the framework lies in its trainable and plug and play world model module. This integration facilitates fidelity imagination rollouts that allow the system to transcend the frequency constraints of physical simulators while significantly enhancing sample efficiency. Furthermore AcceRL achieves superior performance by leveraging a fully decoupled architecture for rollout and inference and training. This design effectively eliminates systemic bottlenecks such as synchronization overhead and memory constraints. By ensuring high hardware utilization and effectively masking communication overheads, it maximizes the benefits of distributed memory optimizations, unlocking near-linear scalability for rollouts and super-linear throughput for the trainer. By simultaneously improving computational throughput and online sample efficiency, AcceRL addresses key system bottlenecks relevant to physical robotic learning, including data collection cost and wall-clock training time. While our current validation is conducted in simulation, these results point toward a scalable foundation for future real-world robotic RL. Future work will focus on deploying the framework for direct real-world reinforcement learning and extending the architecture to accommodate complex agentic training.

References

- [1] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. LIBERO: benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems 36(NeurIPS'2023)*, New Orleans, LA, USA, 2023.
- [2] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong T. Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, Michael S. Ryoo, Krista Reymann, Kanishka Rao, Karl Pertsch, Igor Mordatch, Henryk Michalewski, Yao Lu, Sergey Levine, Lisa Lee, Tsang-Wei Edward Lee, Isabel Leal, Yuheng Kuang, Dmitry Kalashnikov, Ryan Julian, Nikhil J. Joshi, Alex Irpan, Brian Ichter, Jasmine Hsu, Alexander Herzog, Karol Hausman, Keerthana Gopalakrishnan, Chuyuan Fu, Pete Florence, Chelsea Finn, Kumar Avinava Dubey, Danny Driess, Tianli Ding, Krzysztof Marcin Choromanski, Xi Chen, Yevgen Chebotar, Justice Carbajal, Noah Brown, Anthony Brohan, Montserrat Gonzalez Arenas, and Kehang Han. RT-2: vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the Conference on Robot Learning(CoRL'2023)*, pages 2165–2183, Atlanta, GA, USA, 2023.
- [3] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Paul Foster, Pannag R. Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. Openvla: An open-source vision-language-action model. In *Proceedings of the Conference on Robot Learning(CoRL'2024)*, pages 2679–2713, Munich, Germany, 2024.
- [4] Dibya Ghosh, Homer Rich Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, You Liang Tan, Lawrence Yunliang Chen, Quan Vuong, Ted Xiao, Pannag R. Sanketi, Dorsa Sadigh, Chelsea Finn, and Sergey Levine. Octo: An open-source generalist robot policy. In Dana Kulic, Gentiane Venture, Kostas E. Bekris, and Enrique Coronado, editors, *Proceedings of the Robotics: Science and Systems XX (RSS'2024)*, Delft, The Netherlands, 2024.
- [5] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Proceedings of the Robotics: Science and Systems XIX(RSS'2023)*, Daegu, Republic of Korea, 2023.
- [6] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of the Robotics: Science and Systems (RSS'2023)*, Daegu, Republic of Korea, 2023.
- [7] Eric Liang, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael I. Jordan, and Ion Stoica. Rllib: Abstractions for distributed reinforcement learning. In *Proceedings of the 35th International Conference on Machine Learning (ICML'2018)*, pages 3059–3068, Stockholmssmässan, Stockholm, Sweden, 2018.
- [8] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elilob, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging AI applications. In Andrea C. Arpaci-Dusseau and Geoff Voelker, editors, *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation(OSDI'2018)*, pages 561–577, Carlsbad, CA, USA, 2018.
- [9] Zhong Guan, Haoran Sun, Yongjian Guo, Shuai Di, Xiaodong Bai, Jing Long, Tianyun Zhao, Mingxi Luo, Chen Zhou, Yucheng Guo, Qiming Yang, Wanting Xu, Wen Huang, Yunxuan Ma, Hongke Zhao, Likang Wu, Xiaotie Deng, Xi Xiao, Sheng Wen, Yicheng Gong, and Junwu Xiong. RL-vla³: Reinforcement learning vla accelerating via full asynchronism. *arXiv:2602.05765*, 2026.
- [10] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy P. Lillicrap. Mastering diverse domains through world models. *arXiv:2301.04104*, 2023.
- [11] Jake Bruce, Michael D. Dennis, Ashley Edwards, Jack Parker-Holder, Yuge Shi, Edward Hughes, Matthew Lai, Aditi Mavalankar, Richie Steigerwald, Chris Apps, Yusuf Aytar, Sarah Bechtle, Feryal M. P. Behbahani, Stephanie C. Y. Chan, Nicolas Heess, Lucy Gonzalez, Simon Osindero, Sherjil Ozair, Scott E. Reed, Jingwei Zhang, Konrad Zolna, Jeff Clune, Nando de Freitas, Satinder Singh, and Tim Rocktäschel. Genie: Generative interactive environments. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Proceedings of the International Conference on Machine Learning(ICML'2024)*, pages 4603–4623, Vienna, Austria, 2024.
- [12] Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos J. Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38(NeurIPS'2024)*, Vancouver, BC, Canada, 2024.

- [13] Chengxuan Lu, Zhenquan Zhang, Shukuan Wang, Qunzhi Lin, Baigui Sun, and Yang Liu. GIPO: gaussian importance sampling policy optimization. *arXiv:2603.03955*, 2026.
- [14] Jian Hu, Xibin Wu, Weixun Wang, Xianyu, Dehao Zhang, and Yu Cao. Openrlhf: An easy-to-use, scalable and high-performance RLHF framework. *arXiv:2405.11143*, 2024.
- [15] Zhewei Yao, Reza Yazdani Aminabadi, Olatunji Ruwase, Samyam Rajbhandari, Xiaoxia Wu, Ammar Ahmad Awan, Jeff Rasley, Minjia Zhang, Conglong Li, Connor Holmes, Zhongzhu Zhou, Michael Wyatt, Molly Smith, Lev Kurilenko, Heyang Qin, Masahiro Tanaka, Shuai Che, Shuaiwen Leon Song, and Yuxiong He. Deepspeed-chat: Easy, fast and affordable RLHF training of chatgpt-like models at all scales. *arXiv:2308.01320*, 2023.
- [16] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient RLHF framework. In *Proceedings of the European Conference on Computer Systems (EuroSys’2025)*, pages 1279–1297, Rotterdam, The Netherlands, 2025.
- [17] Zhixin Wang, Tianyi Zhou, Liming Liu, Ao Li, Jiarui Hu, Dian Yang, Jinlong Hou, Siyuan Feng, Yuan Cheng, and Yuan Qi. Distflow: A fully distributed RL framework for scalable and efficient LLM post-training. *arXiv:2507.13833*, 2025.
- [18] Lei Zhao, Zihao Ma, Boyu Lin, Yuhe Liu, Wenjun Wu, and Lei Huang. Rllava: An rl-central framework for language and vision assistants. *arXiv:2512.21450*, 2025.
- [19] Hongzhi Zang, Mingjie Wei, Si Xu, Yongji Wu, Zhen Guo, Yuanqing Wang, Hao Lin, Peihong Wang, Liangzhi Shi, Yuqing Xie, Zhexuan Xu, Zhihao Liu, Kang Chen, Wenhao Tang, Quanlu Zhang, Weinan Zhang, Chao Yu, and Yu Wang. Rlinf-vla: A unified and efficient framework for reinforcement learning of vision-language-action models. *arXiv:2510.06710*, 2026.
- [20] James Cipar, Qirong Ho, Jin Kyu Kim, Seunghak Lee, Gregory R. Ganger, Garth Gibson, Kimberly Keeton, and Eric P. Xing. Solving the straggler problem with bounded staleness. In *Proceedings of the 14th Workshop on Hot Topics in Operating Systems (HotOS XIV’2013)*, Santa Ana Pueblo, New Mexico, USA, 2013.
- [21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.
- [22] Guangming Sheng, Yuxuan Tong, Borui Wan, Wang Zhang, Chaobo Jia, Xibin Wu, Yuqi Wu, Xiang Li, Chi Zhang, Yanghua Peng, Haibin Lin, Xin Liu, and Chuan Wu. Laminar: A scalable asynchronous RL post-training framework. In *Proceedings of the 21st European Conference on Computer Systems (EuroSys’2026)*, pages 400–422, Edinburgh, Scotland, UK, 2026.
- [23] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv:2505.24298*, 2025.
- [24] Bo Wu, Sid Wang, Yunhao Tang, Jia Ding, Eryk Helenowski, Liang Tan, Tengyu Xu, Tushar Gowda, Zhengxing Chen, Chen Zhu, Xiaocheng Tang, Yundi Qian, Beibei Zhu, and Rui Hou. Llamarl: A distributed asynchronous reinforcement learning framework for efficient large-scale LLM training. *arXiv:2505.24034*, 2025.
- [25] Weixun Wang, Shaopan Xiong, Gengru Chen, Wei Gao, Sheng Guo, Yancheng He, Ju Huang, Jiaheng Liu, Zhendong Li, Xiaoyang Li, Zichen Liu, Haizhou Zhao, Dakai An, Lunxi Cao, Qiyang Cao, Wanxi Deng, Feilei Du, Yiliang Gu, Jiahe Li, Xiang Li, Mingjie Liu, Yijia Luo, Ziheng Liu, Yadao Wang, Pei Wang, Tianyuan Wu, Yanan Wu, Yuheng Zhao, Shuaibing Zhao, Jin Yang, Siran Yang, Yingshui Tan, Huimin Yi, Yuchi Xu, Yujin Yuan, Xingyao Zhang, Lin Qu, Wenbo Su, Wei Wang, Jiamang Wang, and Bo Zheng. Reinforcement learning optimization for large-scale learning: An efficient and user-friendly scaling library. *arXiv:2506.06122*, 2025.
- [26] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33 (NeurIPS’2020)*, virtual, 2020.
- [27] Zhennan Jiang, Shangqing Zhou, Yutong Jiang, Zefang Huang, Mingjie Wei, Yuhui Chen, Tianxing Zhou, Zhen Guo, Hao Lin, Quanlu Zhang, Yu Wang, Haoran Li, Chao Yu, and Dongbin Zhao. Wovr: World models as reliable simulators for post-training VLA policies with RL. *arXiv:2602.13977*, 2026.
- [28] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC’2020)*, page 20, Virtual, 2020.

- [29] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In Yoshua Bengio and Yann LeCun, editors, *Proceedings of the International Conference on Learning Representations (ICLR'2016)*, San Juan, Puerto Rico, 2016.
- [30] Tongzhou Mu, Zhan Ling, Fanbo Xiang, Derek Yang, Xuanlin Li, Stone Tao, Zhiao Huang, Zhiwei Jia, and Hao Su. Maniskill: Generalizable manipulation skill benchmark with large-scale demonstrations. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*, 2021.
- [31] Hugging Face. Deepspeed integration – accelerate documentation (v0.11.0). <https://huggingface.co/docs/accelerate/v0.11.0/deepspeed>, 2022. Accessed: 2025-12-07.
- [32] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv: 2502.19645*, 2025.
- [33] Haozhan Li, Yuxin Zuo, Jiale Yu, Yuhao Zhang, Zhaohui Yang, Kaiyan Zhang, Xuekai Zhu, Yuchen Zhang, Tianxing Chen, Ganqu Cui, Dehui Wang, Dingxiang Luo, Yuchen Fan, Youbang Sun, Jia Zeng, Jiangmiao Pang, Shanghang Zhang, Yu Wang, Yao Mu, Bowen Zhou, and Ning Ding. Simplevla-rl: Scaling VLA training via reinforcement learning. *arXiv:2509.09674*, 2025.
- [34] Niket Agarwal, Arslan Ali, Maciej Bala, Yogesh Balaji, Erik Barker, Tiffany Cai, Prithvijit Chattopadhyay, Yongxin Chen, Yin Cui, Yifan Ding, Daniel Dworakowski, Jiaojiao Fan, Michele Fenzi, Francesco Ferroni, Sanja Fidler, Dieter Fox, Songwei Ge, Yunhao Ge, Jinwei Gu, Siddharth Gururani, Ethan He, Jiahui Huang, Jacob Samuel Huffman, Pooya Jannaty, Jingyi Jin, Seung Wook Kim, Gergely Klár, Grace Lam, Shiyi Lan, Laura Leal-Taixé, Anqi Li, Zhaoshuo Li, Chen-Hsuan Lin, Tsung-Yi Lin, Huan Ling, Ming-Yu Liu, Xian Liu, Alice Luo, Qianli Ma, Hanzi Mao, Kaichun Mo, Arsalan Mousavian, Seungjun Nah, Sriharsha Niverty, David Page, Despoina Paschalidou, Zeeshan Patel, Lindsey Pavao, Morteza Ramezanali, Fitsum Reda, Xiaowei Ren, Vasanth Rao Naik Sabavat, Ed Schmerling, Stella Shi, Bartosz Stefaniak, Shitao Tang, Lyne Tchaptmi, Przemek Tredak, Wei-Cheng Tseng, Jibin Varghese, Hao Wang, Haoxiang Wang, Heng Wang, Ting-Chun Wang, Fangyin Wei, Xinyue Wei, Jay Zhangjie Wu, Jiashu Xu, Wei Yang, Yen-Chen Lin, Xiaohui Zeng, Yu Zeng, Jing Zhang, Qinsheng Zhang, Yuxuan Zhang, Qingqing Zhao, and Artur Zólkowski. Cosmos world foundation model platform for physical AI. *arXiv:2501.03575*, 2025.

A Key Contributions of AcceRL

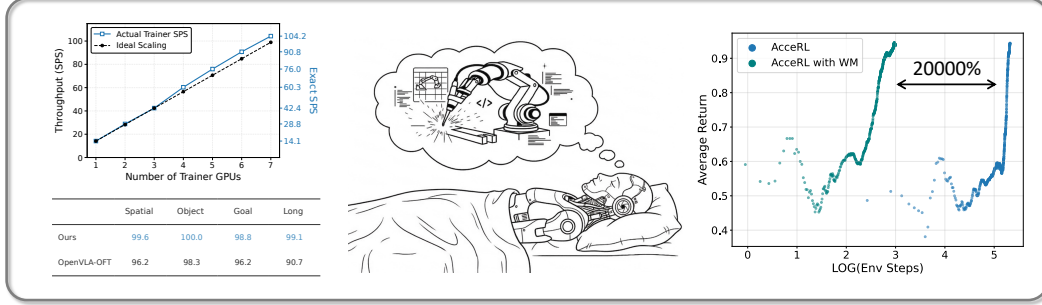


Figure 5: **Overview of AcceRL.** (Top Left) The framework’s throughput exhibits super-linear scaling with the number of trainer GPUs, enabled by ZeRO [28] optimizations. (Bottom Left) Experimental results on the LIBERO benchmark demonstrate that AcceRL achieves highly competitive performance across all evaluation categories, attaining the best results among the compared methods on Spatial, Object, and Long tasks. (Middle) A conceptual illustration of the world model in reinforcement learning, representing the agent “learning in imagination” to significantly boost data utilization. (Right) Performance comparison between model-based (AcceRL with WM) and model-free (AcceRL) approaches. By leveraging a world model pre-trained on 1,000 offline trajectories, the model-based architecture improves online sample efficiency by 2000%.

B Timeline Comparison of synchronous RL framework and asynchronous framework AcceRL

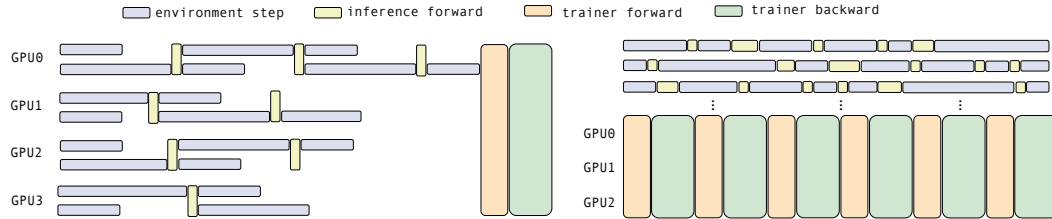


Figure 6: **Timeline of a synchronous RL framework (left) and the asynchronous framework AcceRL (right).** The inference forward blocks in AcceRL encompass both the batching wait time and the inference time. By eliminating the GPU bubbles, AcceRL maximizes hardware utilization and significantly enhances overall training efficiency.

C More Details about Addressing Policy Lag

In an asynchronous distributed architecture, a significant challenge is policy lag, where the behavior policy μ used during rollout becomes stale compared to the current learner policy π . To mitigate the resulting off-policy bias and stabilize convergence, the TrainerWorker implements Value Recomputation, Global Advantage Normalization, and Gradient Calibration.

C.1 Theoretical Equivalency and Implementation Details of Value Recomputation

Deterministic Micro-Batch Slicing Regarding the rigor of deterministic micro-batch slicing, considering that the system employs a gradient accumulation mechanism, the model weights remain frozen until the processing of the entire large batch is complete. According to the linear superposition property of gradients, the gradient of the total objective function can be expressed as the cumulative

sum of the gradients of each micro-batch:

$$\nabla_{\theta} \mathcal{J}_{total} = \sum_{i=1}^{N_{acc}} \nabla_{\theta} \mathcal{J}_{mini_batch}^{(i)} \quad (7)$$

where N_{acc} represents the number of accumulation steps. Because model parameters remain fixed within each gradient-accumulation window, reusing the training forward pass for GAE computation does not introduce additional parameter staleness. Given the same set of trajectory samples before each optimizer step, sequential micro-batch slicing preserves the same large-batch objective under exact arithmetic while avoiding redundant shuffling and value-forward computation.

Communication-Hiding Lag Normalization Regarding lag normalization for communication hiding, the system uses statistics from the previous step to standardize the advantage function:

$$\hat{A}_t = \frac{A_t - \mu_{A,t-1}}{\sigma_{A,t-1} + \epsilon} \quad (8)$$

where $\mu_{A,t-1}$ and $\sigma_{A,t-1}$ denote the global mean and standard deviation of advantages from the previous optimization step. This design provides a practical approximation for hiding the synchronization latency of global advantage normalization. Rather than blocking training to wait for the current batch statistics, AcceRL uses the global statistics from the previous optimization step. The approximation assumes that adjacent-batch advantage statistics change smoothly in practice, which is encouraged by bounded policy updates and large distributed batches but is not theoretically guaranteed. Empirically, we observe that this delayed normalization introduces limited discrepancy while preserving stable convergence.

Engineering Implementation At the engineering implementation level, low-overhead just-in-time Generalized Advantage Estimation (GAE) requires precise handling of boundary states after the micro-batch forward pass. If the sampled data is an unterminated episode, to compute the GAE, the state value of the last time step serves solely as a bootstrap target. Its corresponding loss function is forcibly set to zero to block gradient backpropagation, and the target value node must be detached from the computation graph. During the asynchronous global synchronization phase, the system silently records the sum and the sum of squares of the statistics for the current batch within the local computation loop. Once the synchronization operations across compute nodes are completed at the gradient accumulation boundary, the system uniformly updates the global sliding state using Welford’s algorithm. This ensures the numerical stability of streaming variance calculations and makes them available for the next time step.

C.2 Global Advantage Normalization

To ensure training stability across the distributed training architecture, AcceRL implements a global normalization process after advantage recomputation. Since sampling noise and task difficulties vary across different nodes, utilizing global statistics is essential to eliminate local biases and provide a unified baseline for policy updates.

Instead of performing costly global data shuffling, we efficiently aggregate local statistics (sum, squared sum, and element count) from all nodes via a single distributed AllReduce operation. The pseudo-code for our communication-efficient normalization is as follows:

```
import torch
import torch.distributed as dist

def global_advantage_norm(adv, eps=1e-8):
    # 1. Compute local statistics
    local_sum = adv.sum()
    local_sqr_sum = (adv ** 2).sum()
    local_count = torch.tensor(
        adv.numel(), device=adv.device, dtype=adv.dtype
    )

    # 2. Pack tensors for a single communication call
    stats = torch.stack([local_sum, local_sqr_sum, local_count])
```

```

# 3. Synchronous AllReduce across all nodes
dist.all_reduce(stats, op=dist.ReduceOp.SUM)

# 4. Unpack and compute global mean and standard deviation
global_sum, global_sqr_sum, global_count = stats
global_mean = global_sum / global_count
global_var = (global_sqr_sum / global_count) - (global_mean ** 2)
global_std = torch.sqrt(torch.clamp(global_var, min=0.0))

# 5. Standardize advantages
return (adv - global_mean) / (global_std + eps)

```

The standardized advantage $\hat{A}_t$ is subsequently used for policy gradient computation, ensuring stable optimization signals and improving convergence robustness in high-dimensional manipulation tasks.

D Implementation Details and Refinements for VLA RL

D.1 Vocabulary Slimming

To efficiently adapt the OpenVLA-OFT [32] (based on the Llama-2-7B language backbone) for RL, we prune the language model head lm_head to output only the action vocabulary (e.g., 256 bins), decoupling the policy from the full language lexicon. This reduces VRAM and compute costs while preventing probability mass from leaking to non-action tokens.

AcceRL employs an in-place replacement strategy to crop the original, massive output layer lm_head into a compact linear layer containing only action-related tokens. Specifically, the system first locates the start index and end index of the action tokens in the original vocabulary, then clones the corresponding weights from the original $lm_head.weight$ via slicing. Finally, a new linear layer with an output dimension of $N_{actions}$ is created, and the extracted weights are migrated to this layer, directly overwriting the original language model head.

By slimming the vocabulary, the model only needs to output action commands and does not need to process language text, provides the following advantages:

- **VRAM Efficiency:** Reducing the weight matrix from $[N_{vocab}, d_{model}]$ (where $N_{vocab} \approx 32k$) to $[N_{actions}, d_{model}]$ significantly reduces the existing occupancy of gradients and optimizer states.
- **Increased throughput:** Eliminating most of the computational overhead of the last layer significantly speeds up the iteration frequency of each training step.
- **Noise Reduction:** Physically isolating non-action tokens prevents the model from assigning probability to irrelevant vocabulary, focusing the policy distribution and speeding up convergence.

D.2 Value Head Design

The Value Head is designed to estimate the state value $V(o_t)$ by focusing on decision-critical information within the VLA’s hidden states. In AcceRL, we implement an Action-Aware Attention Pooling mechanism. Given a sequence of all action-related hidden states extracted from the last hidden layer of the VLA, we compute an attention score for each token. These hidden states are detached from the VLA backbone’s computation graph to prevent value gradients from interfering with the policy representation.

The normalized attention weights are then derived via a softmax operation, and the pooled representation is calculated as a weighted sum. Crucially, a state’s value is intrinsically tied to the remaining time horizon; for instance, as the current step approaches the maximum episode limit, the expected future returns drop to zero due to imminent truncation, regardless of the agent’s proximity to success. To explicitly account for this temporal dependency, we incorporate a step embedding that maps the current episode step t to a high-dimensional vector, which is element-wise added to the pooled

feature. The final state value is estimated through a Multi-Layer Perceptron (MLP). The exact tensor operations and dimension transformations are detailed in the PyTorch implementation below:

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ActionAwareValueHead(nn.Module):
    def __init__(self, hidden_dim, max_episode_steps):
        super().__init__()
        # Attention projection
        self.attn_proj = nn.Linear(hidden_dim, 1)

        # Temporal step embedding
        self.step_emb = nn.Embedding(max_episode_steps, hidden_dim)

        # Value estimation MLP
        self.mlp = nn.Sequential(
            nn.Linear(hidden_dim, hidden_dim),
            nn.GELU(),
            nn.Linear(hidden_dim, 1)
        )

    def forward(self, hidden_states, step_t):
        # hidden_states: [Batch, Seq_Len, Hidden_Dim]
        # step_t: Current episode step [Batch]
        # State value V(s_t): [Batch, 1]

        # Detach hidden states from the VLA backbone's computation graph
        h = hidden_states.detach()

        # 1. Compute attention scores and normalize
        # e shape: [Batch, Seq_Len, 1]
        e = self.attn_proj(h)
        # alpha shape: [Batch, Seq_Len, 1]
        alpha = F.softmax(e, dim=1)

        # 2. Compute pooled representation z_pool
        # z_pool shape: [Batch, Hidden_Dim]
        z_pool = torch.sum(alpha * h, dim=1)

        # 3. Incorporate step embedding
        # e_step shape: [Batch, Hidden_Dim]
        e_step = self.step_emb(step_t)

        # 4. Final state value estimation
        # v shape: [Batch, 1]
        v = self.mlp(z_pool + e_step)

        return v
```

D.3 Token-level Policy Optimization

Defining the granularity of the optimization objective is critical for training stability, leading us to distinguish between three primary approaches: chunk-level, action-level, and our adopted token-level optimization.

Standard chunk-level methods treat the entire sequence of K tokens as a single unified action. The joint probability is calculated as $P(a_{1:K}|o) = \prod_{k=1}^K \pi(a_k|o, a_{<k})$, which frequently leads to severe numerical instability. Because the joint probability is a product of multiple values between 0 and 1, the resulting magnitude can become infinitesimally small, causing the PPO importance sampling ratio

$\rho_t(\theta) = \frac{\pi_\theta(a_{1:K}|o)}{\pi_{\theta_{old}}(a_{1:K}|o)}$ to fluctuate wildly. Such fluctuations often trigger the PPO clipping mechanism, leading to a total loss of gradient signals and stalled training.

Alternatively, an action-level approach might compute the loss based on the mean or aggregated probability of the tokens within a single action dimension. However, this still risks blurring the independent learning signals of individual tokens, failing to provide precise credit assignment across the multi-dimensional action space of the VLA.

In contrast, we implement a token-level optimization strategy. By treating each action token as an independent decision point conditioned on the shared observation, the importance sampling ratio is redefined for each individual token k :

$$\rho_{t,k}(\theta) = \frac{\pi_\theta(a_{t,k} | o_t)}{\pi_{\theta_{old}}(a_{t,k} | o_t)} \quad (9)$$

By calculating the clip loss at the token level, we prevent outlier tokens from causing the entire multi-dimensional action’s gradient to be clipped, thereby retaining a much higher density of training signals. This approach ensures numerical robustness by avoiding the vanishing product problem and provides precise credit assignment across specific components of the action space.

D.4 Dynamic Weighted Resampling

To improve sample efficiency and balance the learning progress across diverse tasks, we implement a Dynamic Weighted Resampling (DWR) mechanism. DWR adaptively adjusts the sampling probability of each task based on its recent performance.

Instead of relying on a static uniform distribution, we maintain a sliding window of recent episode outcomes for each task. The sampling weight is dynamically computed proportional to the recent failure rate, complemented by a Laplace smoothing factor:

```
import numpy as np

class DynamicWeightedResampler:
    def __init__(self, num_tasks, window_size=100, eps=1.0):
        self.num_tasks = num_tasks
        self.window_size = window_size
        self.eps = eps

        # Circular buffer for history (1.0 for success, 0.0 for failure)
        # Initialized to ones to prevent early bias against unattempted tasks
        self.history = np.ones((num_tasks, window_size))
        self.ptr = 0

    def update_history(self, task_idx, success_flag):
        self.history[task_idx, self.ptr] = success_flag
        self.ptr = (self.ptr + 1) % self.window_size

    def sample_task(self):
        # 1. Count failures (Total window size - successes)
        success_counts = np.sum(self.history, axis=1)
        failure_counts = self.window_size - success_counts

        # 2. Apply Laplace smoothing to compute sampling weights
        weights = failure_counts + self.eps

        # 3. Normalize to obtain sampling probabilities
        probabilities = weights / np.sum(weights)

        # 4. Sample task index based on the generated distribution
        return np.random.choice(self.num_tasks, p=probabilities)
```

This DWR strategy effectively directs computational resources toward challenging tasks where performance lags. Furthermore, the smoothing term eps guarantees that all tasks maintain a non-zero

sampling probability, effectively preventing catastrophic forgetting of previously mastered skills while the agent tackles new challenges.

D.5 Asynchronous Parallel Data Prefetching

To address the I/O bottlenecks inherent in experience replay sampling and trajectory processing, we implement a data prefetching architecture based on asynchronous parallel logic. By maintaining a background asynchronous loop within the trainer that operates independently of the primary training task, AcceRL achieves an effective decoupling of data preparation from gradient updates. Functioning as a producer, this background process continuously monitors the distributed replay buffer and triggers cross node trajectory sampling once the required threshold is met. This concurrency ensures that while the GPU executes gradient computations for the current batch, system resources are simultaneously preparing the data for the subsequent iteration. AcceRL simplifies the training critical path by delegating observation tensorization and batching operations to the background prefetcher. The trainer extracts ready to use super-batches directly from a local cache. This streamlined data loading mechanism effectively significantly reduces I/O latency, maximizing hardware utilization and accelerating convergence in complex embodied tasks.

D.6 High-Performance NCCL-based Weight Synchronization

The performance of an online RL framework highly depends on the efficiency of policy synchronization between the trainer and the Inference Worker. If the Inference Worker uses stale weights due to synchronization delays, it will cause severe policy staleness, which may disrupt the stability and sample efficiency of near-on-policy training.

To address this, our framework uses a weight synchronization mechanism based on NCCL collective communication, which upgrades the synchronization path from traditional CPU/TCP or shared storage to the level of GPU-to-GPU collective communication:

1. **Intra-node Synchronization Group Construction:** In a multi-node distributed deployment, the synchronization is organized by group. The trainer-side source and the Inference Worker form an independent NCCL Broadcast Group, ensuring that weight synchronization is handled locally. It fully utilizes the high bandwidth of NVLink and avoids the potential bottlenecks of the cross-node InfiniBand network.
2. **In-place Tensor Update:** Whenever the training side completes a gradient update, the trainer directly broadcasts the model’s trainable parameters and necessary buffers in a fixed sequence to the inference GPU. After the inference GPU receives the tensors, it directly overwrites the inference model weights through in-place operations. This avoids the overhead of Python serialization, host-side memory copying, or object storage distribution.

To further reduce trainer waiting latency at synchronization points, we propose a lightweight Inference Drain mechanism. The trainer sends a drain signal to Inference Worker well before parameter updates finish. Upon receiving this signal, Inferences stop scheduling new forward batches. Instead, they first complete all ongoing computations, then switch to the state for receiving updated weights. This mechanism effectively prevents the trainer from being blocked by the long-duration tail of inference forward computations, while also guaranteeing the atomicity and version consistency of model updates on the inference side. Through this fine-grained synchronization control, the system significantly enhances overall hardware coordination efficiency while maintaining high-frequency policy updates.

E Experiment Setup

The framework consists of three core components for world-model-based reinforcement learning. The policy model M_{policy} utilizes the OpenVLA-OFT [32] architecture to process observation images and proprioceptive states into discretized action tokens and employs action chunking for temporal consistency. The observation model M_{obs} is a diffusion-based transition model built upon the DIAMOND (Diffusion As a Model Of eNvironment Dreams) [12] architecture. Unlike traditional world models that compress environment dynamics into discrete latent variables, DIAMOND leverages a diffusion process to preserve fine-grained visual details. This enables M_{obs} to generate high-fidelity,

synthesized next-frame images conditioned on historical observation sequences and current action chunks. Acting as a virtual referee, the reward model M_{reward} is a binary classifier modified from OpenVLA-OFT that evaluates these synthetic frames to estimate success probabilities and termination signals. In our experiments, we utilize the OpenVLA-OFT backbone and the DIAMOND world model. The OpenVLA codebase is released under the MIT License, while its pre-trained model weights (built upon Llama-2-7B) are distributed under the Llama 2 Community License. DIAMOND and other baseline implementations are used in accordance with the MIT License.

Our experiments were conducted on high-performance compute nodes equipped with NVIDIA H200 (141GB) GPUs, providing the necessary VRAM for efficient VLA fine-tuning. The ManiSkill training runs utilized 4 GPUs for 2.193 days (totaling ~ 210.5 GPU-hours), while the LIBERO benchmark training utilized 8 GPUs for 2 days (totaling ~ 384 GPU-hours). The total compute consumption for the core results is approximately 594.5 H200 GPU-hours.

Table 3: Detailed Experimental Settings and Hyperparameters for AcceRL on ManiSkill tasks.

Category	Value
Environment & Task	
Simulator	ManiSkill (Task: PickCube-v1)
Observation Space	RGB Camera (base_camera, 128×128)
Max Episode Steps	100
Model Architecture	
VLA Backbone	OpenVLA-OFT (based on Llama-2-7B)
Action Vocabulary Size	256 bins (Slimmed)
Value Head	MLP with Action-Aware Attention Pooling
RL Training (GIPO)	
Optimizer	AdamW (with bfloat16 mixed precision)
Learning Rate (Policy / Value)	3×10^{-6} / 3×10^{-5}
Warmup Steps	500
Micro-batch Size	16 per GPU
Gradient Accumulation Steps	2
Total Global Batch Size	96
Discount Factor (γ)	0.99
GAE Lambda (λ)	0.95
GIPO Clip Epsilon	0.2
KL Divergence Coefficient	0.1
Entropy Coefficient	0.0
AcceRL System Settings	
Value Recomputation	Enabled (recompute_value: true)
Trainer GPUs	3
Inference GPUs	1
CPU Rollout Workers	6
Replay Buffer Capacity	3000 episodes
Total Training Iterations	60,000

Table 4: Detailed Experimental Settings and Hyperparameters for OpenVLA-OFT RL with Cosmos WM on LIBERO Object tasks.

Category	Value
Environment & Task	
Simulator / Benchmark	LIBERO
Observation Space	RGB Image (num_images_in_input:2)
Model Architecture	
VLA Backbone	OpenVLA-OFT
Cosmos WM Actions	16
RL Training	
Mixed Precision	bfloat16 (-use-bf16)
Learning Rate (Policy / Value / Reward / Denoiser)	$1 \times 10^{-5} / 1 \times 10^{-4} / 1 \times 10^{-4} / 1 \times 10^{-5}$
Warmup Steps (Policy / Value / Reward / Denoiser)	500 / 500 / 500 / 500
Train Batch Size	8
Gradient Accumulation Steps	16
Effective Trainer Batch Size	128 ($1 \times 8 \times 16$)
Discount Factor (γ)	0.99
GAE Lambda (λ)	0.95
GIPO sigma	0.2
Value Loss Coefficient	0.5
KL Divergence Coefficient	0.1
Maximum KL	0.02
Entropy Coefficient	0.01
World Model & Cosmos Settings	
Imagine Horizon	8
Number of Step Conditions	13
Reward Scale	5.0
Cosmos State t	8
Cosmos Guidance Scale	7.0
Cosmos Sampling Steps	35
Cosmos Number of Returns	8
Denoiser Batch Size	2
Denoiser Accumulation Steps	2
Denoiser Train Interval	3
Reward Batch Size	2
Reward Accumulation Steps	2
Reward Train Interval	10
System Settings	
Trainer GPUs	1
Inference Actors	1
Reward Inference Actors	1
Denoiser Inference Actors	1
Rollout Workers	30
Evaluation Workers	10
Inference Batch Size	8
Total Training Iterations	30,000
Replay Capacity	10,000
World Model Replay Capacity	50,000
Real Trajectory Collect Interval	3

Table 5: Detailed Experimental Settings and Hyperparameters for AcceRL WM on LIBERO tasks.

Category	Value
Environment & Task	
Simulator	LIBERO
Observation Space	RGB Image (num_images_in_input:1)
Model Architecture	
VLA Backbone	OpenVLA-OFT (based on Llama-2-7B)
Action Vocabulary Size	256 bins (Discrete Actions)
RL Training (GIPO)	
Optimizer	AdamW (with bfloat16 mixed precision)
Learning Rate (Policy / Value / Reward /denoiser)	$1 \times 10^{-5} / 1 \times 10^{-4} / 5 \times 10^{-5} / 1 \times 10^{-4}$
Warmup Steps (Policy / Value)	500 / 500
Micro-batch Size	16 per GPU
Gradient Accumulation Steps	40
Total Global Batch Size	1280 ($2 \times 16 \times 40$)
Discount Factor (γ)	0.99
GAE Lambda (λ)	0.95
GIPO sigma	0.2
KL Divergence Coefficient	0.1
Entropy Coefficient	0.00
World Model & System Settings	
Imagine Horizon	2
Number of Step Conditions	4
Reward Train Interval	15
Trainer GPUs	2
Inference Workers	1
CPU Rollout Workers	24
Evaluation Workers	12
Total Training Iterations	30,000

Table 6: Detailed Experimental Settings and Hyperparameters for OpenVLA RL Training.

Category	Value
Environment & Task	
Benchmark	LIBERO
Observation Space	RGB images + proprioceptive state
Number of Images in Input	2
Model Architecture	
VLA Backbone	OpenVLA-OFT
Action Space	Discrete actions
RL Training (GIPO / PPO)	
Optimizer	AdamW (with bfloat16 mixed precision)
Learning Rate (Policy / Value)	$3 \times 10^{-6} / 3 \times 10^{-5}$
Train Batch Size	8
Gradient Accumulation Steps	16
Total Training Iterations	60,000
Discount Factor (γ)	0.99
GAE Lambda (λ)	0.95
Value Function Coefficient	0.5
KL Divergence Coefficient	0.1
Entropy Coefficient	0.0
GIPO Sigma	0.5
System Settings	
Trainer GPUs	3
Inference Actors	1
CPU Rollout Workers	30
Evaluation Workers	10
Inference Batch Size	8
Replay Buffer Capacity	3000 episodes
Object Store Memory	256 GB
Checkpoint Interval	2,000,000 steps

F Detailed Tabular Data Comparison of Throughput

Table 7: **Scaling performance and hardware utilization of AcceRL.** The data illustrates a super-linear increase in training throughput (SPS) while maintaining consistently high GPU utilization (> 94%) at different scale points.

GPUs	Throughput (SPS)	GPU Util. (%)
1	14.13	96.45
2	28.82	97.17
3	42.42	94.22
4	60.33	98.36
5	75.95	96.72
6	90.78	96.63
7	104.22	95.07

G Ablation Study

G.1 Ablation Study of AcceRL without Value Recomputation

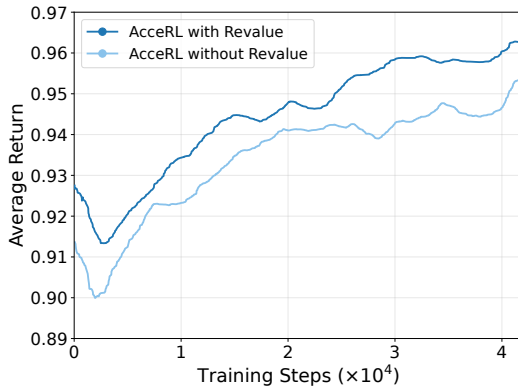


Figure 7: **Ablation study of the AcceRL framework regarding the value recomputation mechanism.** The comparison highlights the critical role of recomputing values in maintaining training stability. AcceRL with Revalue (dark blue) effectively handles data staleness in low-rollout scenarios, while AcceRL without Revalue (light blue) exhibits significant performance drops and higher variance due to misaligned value targets.

While the previous results demonstrate the overall efficiency of the AcceRL framework, its performance in asynchronous distributed settings relies heavily on maintaining consistency between the actor and the critic. A significant technical challenge in such architectures is data staleness: since rollout and training occur in parallel, the experiences stored in the replay buffer were often evaluated by an earlier, outdated version of the critic network. To mitigate the resulting gradient noise and training instability, we introduced a value recomputation mechanism that updates value targets using the most recent critic before each gradient update. As illustrated in Figure 7, the value recomputation mechanism serves as a critical stabilizer for off-policy VLA fine-tuning. The full AcceRL framework (dark blue) effectively handles data staleness, maintaining high average returns with minimal variance even in resource-constrained settings.

In contrast, the version without recomputation (light blue) exhibits performance oscillations and a lower convergence ceiling. This instability arises from misaligned value targets, where the policy is updated based on evaluations that no longer reflect the current state of the critic. By ensuring that the policy is always guided by the most accurate and up-to-date value estimates, the recomputation mechanism prevents error accumulation and ensures robust policy improvement throughout the training process.

G.2 Ablation Study of Gaussian Importance sampling Policy Optimization

Beyond system-level optimizations, the choice of policy optimization objective is critical for stability in asynchronous settings. Given that the inherent policy lag in decoupled architectures can destabilize standard PPO, we employ GIPO [13], which replaces hard clipping with a Gaussian trust weight to softly damp extreme importance ratios.

As illustrated in Figure 8, we plot the raw evaluation returns to explicitly visualize the training dynamics. A prominent, rapid surge is observed at the very beginning of the training (near 0k steps). This phenomenon is a statistical artifact inherent to the Episode-Level Long-Tail in our asynchronous rollout mechanism. During the initial phase, successful trajectories typically have shorter horizons and are therefore completed and reported to the buffer faster than unsuccessful ones that persist through the full task duration [20]. This leads to a temporary, optimistic skew in success rates before the system reaches a representative sample distribution.

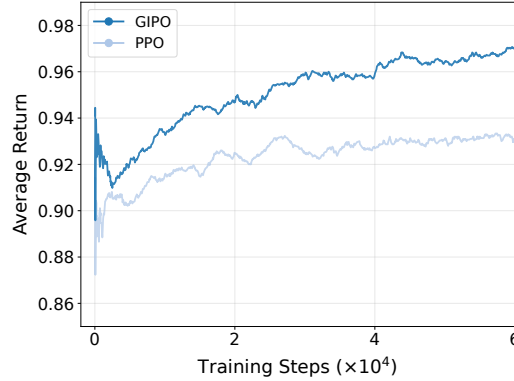


Figure 8: **Ablation study of the AcceRL framework regarding the GIPO algorithm.** The curves highlight the stability of GIPO (dark blue) compared to standard PPO (light blue). GIPO maintains a robust, high-performance trajectory, whereas PPO exhibited oscillatory behavior and inferior performance due to policy lag.

Once stabilized, the comparison reveals a clear divergence. The AcceRL framework equipped with GIPO (dark blue) maintains a tightly clustered, high-performance trajectory. In contrast, the AcceRL variant utilizing standard PPO (light blue) exhibits severe performance oscillations and significantly lower sample efficiency. This instability is primarily due to PPO’s reliance on a hard clipping mechanism, which becomes counterproductive in asynchronous settings where data generation can be delayed or experiences may become stale. In such scenarios, a large proportion of the sampled data triggers the clipping threshold, resulting in ineffective stale updates with zero gradients that fail to improve the policy.

To overcome these algorithmic limitations, GIPO replaces hard clipping with a smooth log-ratio trust-weighted surrogate. This design inherently provides a symmetric and continuous penalty, implicitly enforcing a tunable bound on the policy update magnitude without abruptly zeroing out useful gradients. Furthermore, by achieving a superior bias-variance trade-off, GIPO provides formal guarantees of robustness and stability under finite-sample estimation, even when learning from highly stale replay data. By effectively mitigating this data utilization collapse, GIPO demonstrates a dramatic acceleration in convergence: it achieves a performance level at approximately 8,000 steps that standard PPO requires 60,000 steps to reach, representing a roughly 7.5-fold improvement in sample efficiency.

G.3 Ablation Study of the Trainer-Infer Weight Synchronization Mechanism

To analyze the impact of synchronization efficiency on system performance, we compare our framework’s NCCL collective communication scheme with two other common synchronization paths in distributed reinforcement learning:

1. **Host-Mediated PCIe Transfer:** This approach is also known as CPU-staged GPU memory transfer. In a typical workflow, the trainer copies weights from GPU memory to CPU host memory. These weights are then distributed to inference nodes via a parameter server, Ray Object Store, or underlying TCP paths. Finally, the Inference Workers copy the weights from host memory back to their GPU memory. In this path, PCIe bandwidth, object management overhead, and potential Python serialization become the primary bottlenecks.
2. **Shared-Storage Checkpoint Reload:** Methods like AReal couple policy synchronization with checkpoint management. After completing a training step, the trainer saves the model weights to a shared disk (such as NFS or a parallel file system). The inference servers then use polling signals to trigger a disk read and reload the model.

Our experiments are conducted on four 8-GPU H200 nodes, with the 7B OpenVLA-OFT as the backbone model. We allocate 6 GPUs for training and 2 GPUs for inference.

Table 8: **Comparison of synchronization overheads across different methods.** By employing NCCL with an early drain strategy, our system significantly reduces synchronization latency, infer update delay, and sample policy lag compared to traditional host-mediated or shared-storage approaches.

Method	Sync Latency	Infer Update Delay	Sample Policy Lag
NCCL with drain	70–95 ms	70–95 ms	0.03–0.05
NCCL without drain	220–300 ms	220–300 ms	0.1–0.2
PCIe / Host-mediated	2–7 s	3–12 s	0.3–1.5
Shared-storage / AReal-style	12–53 s	17–76 s	2–12

Experimental results demonstrate the advantages of the NCCL-based synchronization mechanism for online training of ultra-large-scale VLA models. By eliminating redundant disk I/O and CPU transfers, this approach successfully reduces the update latency for 7B-scale models from tens of seconds to a sub-second level of under 100 milliseconds, achieving a performance improvement of two orders of magnitude. Combined with inference drain, the system further optimizes the trainer’s waiting overhead at synchronization points, significantly reducing the sync wait time from 220–300 ms to 70–95 ms. This fine-grained pipeline alignment ensures high throughput on the training side. Another impact is reflected in the significant reduction of policy staleness. Experiments show that a more efficient communication path directly reduces the Sample Policy Lag, which drops from 2–12 under the shared storage scheme to 0.03–0.05. This means rollout data generated by Inference Workers can quickly reflect the policy distribution currently optimized by the trainer. This fast feedback is critical for large-scale VLA models to maintain numerical stability and convergence efficiency in complex physical simulation tasks. Furthermore, compared to AReal, which suffer from global barrier blocking and severe long-tail I/O latency during synchronous writes, our NCCL synchronization scheme aligns perfectly with the distributed training pipeline. Even in high-frequency synchronization scenarios, the proposed mechanism exhibits excellent system scalability, with nearly negligible impact on overall training throughput.

G.4 Comprehensive Multi-Seed Evaluation

To rigorously evaluate the statistical significance and competitive upper bound of our proposed mechanisms, we conducted an extensive multi-seed evaluation across 10 MuJoCo environments. Benchmarking five algorithmic variants—standard PPO, SAPO, and GIPO with varying base trust region widths ($\sigma \in \{0.2, 0.5, 1.0\}$)—we collected valid trajectories from 196 successful independent runs (accounting for minor cluster preemptions out of 200 theoretical runs). Following the rigorous protocols in *RLiable*, we report the Interquartile Mean (IQM) and Mean Normalized Scores based on the final 50 evaluation points of the training process. The IQM provides a robust aggregate metric by discarding the highest and lowest 25% of runs, effectively filtering out anomalous seed performances to reveal the algorithm’s stable learning capability.

Table 9: **Aggregate Performance Scores.** Evaluated across 196 independent runs in 10 MuJoCo environments. GIPO(0.2, 1.0) achieves a substantial lead in both IQM (robustness) and Mean Normalized score (overall average).

Rank	Algorithm	IQM	Mean Norm.
1	GIPO(0.2, 1.0)	0.724	0.651
2	PPO	0.595	0.565
3	GIPO(0.5, 1.0)	0.578	0.582
4	GIPO(1.0, 1.0)	0.446	0.481
5	SAPO	0.315	0.366

As detailed in Table 9, GIPO(0.2, 1.0) establishes a commanding lead across all metrics, achieving an IQM of 0.724 and a Mean Normalized score of 0.651. This creates a substantial performance gap over the baseline PPO (0.595 IQM) and completely outperforms SAPO (0.315 IQM).

These comprehensive results provide critical insight into asynchronous training dynamics: under conditions of high data staleness, a narrower foundational trust region ($\sigma = 0.2$) serves as an essential stabilizer. While standard PPO struggles with policy lag due to hard clipping, and larger σ values (e.g., 1.0) allow excessive divergence, GIPO’s tightly bounded Gaussian trust weight filters out heavily delayed off-policy data without abruptly zeroing out all gradients. This strict yet smooth attenuation prevents catastrophic policy degradation, enabling GIPO to consistently learn high-performing policies even in challenging, delayed-feedback distributed architectures.

H Broader Impacts

Our work on the AcceRL framework introduces significant efficiency improvements for training Vision-Language-Action (VLA) models. A primary positive impact is the democratization of embodied AI research; by significantly reducing the computational resources and time required to train large-scale models, we lower the barrier to entry for researchers with limited compute budgets, which also helps reduce the carbon footprint associated with heavy GPU training. Regarding potential negative impacts, accelerating the development of highly capable robotic policies could inadvertently expedite labor automation, potentially leading to workforce displacement in certain sectors. Furthermore, as with any embodied AI system, deploying these models in real-world physical environments without rigorous safety guardrails poses inherent physical risks.